UPI Guard: An Audited Pre-Debit Fraud-Risk Benchmark for UPI Payments with Learnability Testing, Out-of-Time Evaluation and INR Cost Analysis

Yogendra Pandey, Sahil Mangla, Kuldeep Khengar
Goa Institute of Management — Big Data Analytics
Student IDs: B2026109, B2026097, B2026082
Email: {yogendra.pandey, sahil.mangla, kuldeep.devendrasinhkhengar}2026b@gim.ac.in

***Abstract*—Real-time fraud screening for India's Unified Payments Interface (UPI) must decide, before a debit is authorised, whether to approve a payment or demand step-up verification. We build and evaluate such a pre-debit scorer on a public log of 250,000 UPI transactions from calendar year 2024 (480 labelled frauds, 0.192% prevalence). Unlike our earlier drafts, every number reported here comes from one identified, seeded run, whose dataset hash, split counts and confusion counts are published. The pipeline (i) removes the post-authorisation `transaction_status` field and uses only 24 features available before debit, including past-only velocity counts; (ii) separates training (Jan–Aug), calibration and threshold selection (Sep–Oct) and a single out-of-time test (Nov–Dec; 41,002 transactions, 77 frauds); (iii) audits whether the label is learnable at all; and (iv) converts scores into a net benefit in Indian rupees using the real transaction amounts. The audit finds no attribute associated with fraud after Benjamini–Hochberg correction (0 of 18 tests), and a label-permutation test gives $p = 0.448$. Logistic regression, random forest, LightGBM, an MLP and a 1D-CNN reach test ROC-AUC between 0.495 and 0.565. The best, LightGBM at 0.565 (95% CI [0.513, 0.621]), gives no reliable evidence of skill: it is not significant after Holm correction ($p = 0.123$), lies below the minimum detectable AUC of 0.582, and contradicts its own training-window CV AUC of 0.492. At a review cost of INR 25, a flag pays for itself only above 1.66% precision (8.7$\times$ the base rate), so the cost-optimal policy is to intervene on no transaction. A positive control with planted, simulated signal shows that the same pipeline recovers near-oracle discrimination (AUC 0.986 vs. oracle 0.993), so the null result reflects the data, not the method. A feature-order experiment shows that the 1D-CNN's convolutions add nothing. Its accuracy is insensitive to column order, it does not beat order-invariant models, and it trails them on PR-AUC. We conclude that this public dataset cannot support claims of production-grade UPI fraud detection, including the AUC of 0.999 in our earlier drafts, and we specify the data a real deployment would require.**

***Index Terms*—UPI, payment fraud, pre-debit screening, data provenance, out-of-time validation, permutation test, calibration, cost-sensitive decision, positive control, tabular deep learning**

I. Introduction

UPI settles payments within seconds, so a fraudulent debit is rarely recoverable after the fact. The economically relevant intervention is therefore *pre-debit*. At the moment the payer confirms a payment, the payer's bank or payment service provider (PSP) can approve the payment, or impose friction such as an additional authentication factor, a cooling-off delay or a manual review [1], [2]. A useful scorer must (a) use only information that exists at that moment, (b) output probabilities calibrated well enough to compare with a monetary cost, and (c) be validated on a later time period than it was trained on, because fraud patterns drift [4].

Our first-review manuscript reported AUC 0.9991 and equal precision and recall of 97.69% for a 1D-CNN. The review identified three defects. The data provenance was unresolved. The sample sizes conflicted (31,250 versus 625 test rows). The metrics were internally inconsistent: at 0.192% prevalence, recall 97.69% and specificity 99.39% imply a precision of only about 23.6%, not 97.69%. This paper replaces those results with a single traceable experiment. In doing so, it reaches a different and better-supported conclusion.

Contributions.

1) A *feature-availability* protocol for pre-debit UPI screening. It drops authorisation outcomes, fits reference statistics on the training window only, and uses strictly backward-looking velocity counts, all enforced by unit tests (Sec. IV).
2) A *learnability audit*: multiplicity-corrected association tests, a label-permutation test [16] and a power analysis. These should precede any reported fraud model (Sec. V-A).
3) A reconciled, *out-of-time* comparison of six models. It reports bootstrap confidence intervals, exact TP/FP/FN/TN counts, calibration error, and a rupee-denominated net benefit with a break-even precision rule (Secs. IV-C–V-D).
4) A *positive control* with planted signal that validates the pipeline's power. It also includes a controlled test of whether 1D convolutions over tabular columns carry any justified advantage (Sec. V-E).

II. Related Work

Fraud detection. Bolton and Hand [3] frame fraud detection as supervised and unsupervised classification under

extreme imbalance and label delay. Dal Pozzolo *et al.* [4] show that realistic evaluation requires chronological splits and verification-latency-aware feedback. Whitrow *et al.* [6] and Bahnsen *et al.* [7] show that the strongest card-fraud signals are customer-level aggregations, such as transaction velocity, deviation from the customer's own history, and periodic spending patterns. These need a persistent payer identifier.

Synthetic financial data. Public mobile-money and payment datasets are frequently simulated, as with PaySim [8], because real labelled transactions are confidential. Results on such data reflect the simulator's assumptions, so documenting provenance is essential [9], [10].

Evaluation under imbalance. With prevalence near 0.2%, ROC-AUC can look favourable while precision is very low. Precision–recall analysis is more informative [11], [12]. Uncertainty in AUC can be quantified analytically [13], [14] or by bootstrap [15]. Permutation tests establish whether a classifier beats chance [16]. Leakage from features unavailable at decision time is a common source of inflated results [18].

Calibration and cost. Cost-sensitive decisions need calibrated probabilities [19]. Platt scaling [20] and isotonic regression [21] are standard post-hoc methods. Isotonic regression overfits when positives are few [21]. Resampling methods such as SMOTE [22] distort probabilities and must be corrected [5].

Deep learning on tables. Large benchmarks find that tree ensembles such as LightGBM [23] and random forests [24] still match or beat deep networks on typical tabular data [25], [26]. Convolutions assume a meaningful local neighbourhood, which tabular column order does not provide.

III. Data and Provenance

A. Source and integrity

The study uses the file `upi_transactions_2024.csv` (SHA-256 prefix `8e46a45fd12c3e9e`; full hash in the repository manifest). It is a publicly redistributed transaction log with 250,000 rows and 17 columns, dated 2024-01-01 to 2024-12-30. It was *not* released by NPCI, the RBI or any bank. It contains no payer, payee, account or device identifier, and no generating procedure is documented. Integrity checks found 0 missing values, 0 duplicate rows and 0 duplicate IDs. `hour_of_day`, `day_of_week` and `is_weekend` are exactly consistent with `timestamp`. Amounts range from INR 10 to INR 42,099 (median INR 629). 480 rows are labelled fraud (0.192%).

Provenance statement. Earlier drafts described this file both as "empirical" and as "generated". We now state only what can be verified: the file is public, and its origin is undocumented. Sec. V-A shows that its label behaves as if assigned independently of every recorded attribute, which is typical of a simulated dataset. We therefore treat it as a *public benchmark of unknown, probably synthetic, origin*. No statement in this paper concerns real UPI fraud rates or losses.

B. Feature availability at the pre-debit decision

Table I classifies every column by whether it exists before authorisation. `transaction_status` (SUCCESS/FAILED) is the outcome of authorisation. Using it would leak post-decision information [18], so it is excluded from all models. It is retained only in the audit.

TABLE I
Raw columns and their pre-debit availability

Column	Values	Pre-debit use
transaction type	P2P, P2M, Bill, Recharge	Yes
merchant_category	10 categories	Yes
amount (INR)	10–42,099	Yes (log; train-fitted ratios)
sender/receiver age group	5 brackets each	Yes (KYC)
sender_state	10 states	Yes
sender/receiver bank	8 banks each	Yes (VPA lookup)
device_type, network_type	3 / 4 levels	Yes
hour, weekday, weekend	from timestamp	Yes
timestamp	2024	Ordering, split, velocity
transaction_status	SUCCESS/FAILED	**No**: post-authorisation
transaction id	unique	No: identifier
fraud_flag	0/1	Target

TABLE II
Chronological split used in the reported run

Split	Period	Rows	Frauds	Legit.	Rate (%)
Train	2024-01-01 – 2024-08-31	167,149	316	166,833	0.189
Validation	2024-09-01 – 2024-10-31	41,849	87	41,762	0.208
Test	2024-11-01 – 2024-12-30	41,002	77	40,925	0.188
Total		250,000	480	249,520	0.192

C. Out-of-time split

Rows are ordered by timestamp and split by calendar date (Table II). The test window is scored exactly once, after all modelling choices are frozen.

IV. Methodology

A. Pre-debit feature engineering

Let transaction i occur at time t_i with amount a_i in merchant category c_i. We construct 24 inputs, grouped as follows.

Transaction: $\log(1 + a_i)$; the ratio $a_i/\tilde{a}_{c_i}$ to the training-window median of the category; the training-window percentile of a_i within c_i; and an indicator $a_i > q_{0.95}(c_i)$. All reference statistics come from the training window only.

Temporal: hour, $\sin(2\pi h/24)$, $\cos(2\pi h/24)$, a night flag (00:00–05:59), weekend, and weekday.

Party: sender and receiver age groups, their absolute ordinal gap, sender state, sender and receiver banks, and a same-bank flag.

Device: device and network type.

Velocity: because no payer identifier exists, we use segment-level counts. The segment is g_i = (sender bank, state, age group, device). The counts are

$$v_i^{(w)} = \left|\{\, j : g_j = g_i,\ t_i - w < t_j < t_i \,\}\right|, \qquad (1)$$

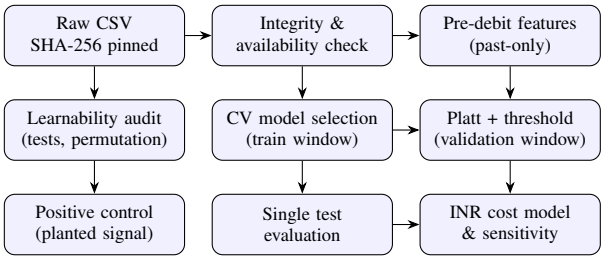


Fig. 1. UPI Guard pipeline. Each box corresponds to a module in the released code.

for $w \in \{1\,\text{h}, 24\,\text{h}\}$, plus a global 10-minute count. The strict inequality $t_j < t_i$ guarantees that only the past is used. A unit test confirms that appending future rows never changes a past row's features. No feature uses labels.

B. Learnability audit

For each attribute, we test independence from the label: Pearson χ^2 for categorical attributes (with Cramér's V), and Mann–Whitney U for numeric ones (with the univariate AUC $U/(n_1 n_0)$). The 18 p-values are adjusted with Benjamini–Hochberg [17]. For the joint question, we compute the 3-fold cross-validated AUC of an L_2 logistic regression on all pre-debit attributes. We compare it with the same statistic under 200 random permutations of the label, with $p = (1 + \#\{\text{AUC}^\pi \geq \text{AUC}\})/(1 + 200)$ [16].

Power. Under H_0: AUC $= 0.5$, the standard error of the empirical AUC is $\sqrt{(n_1 + n_0 + 1)/(12 n_1 n_0)}$ [13]. With $n_1 = 77$ test frauds, this is 0.0329. A one-sided 5% test therefore requires AUC > 0.554, and 80% power requires a true AUC of at least **0.582**. Smaller real effects cannot be confirmed on one two-month test window.

C. Models and selection

We compare a prevalence baseline (constant prior), L_2 logistic regression (one-hot inputs, balanced class weights), a random forest [24] (300 trees, balanced bootstrap weights), LightGBM [23], an MLP (64–32 units, early stopping), and a 1D-CNN that re-implements our earlier architecture. The CNN has two `Conv1d` layers (32 and 64 filters, kernel 2) with batch normalisation and ReLU, dropout 0.25, a dense layer of 32 units, and a sigmoid output. It uses positive-class weight $\sqrt{n_0/n_1}$ over the 78 one-hot/scaled inputs. Small, pre-declared grids are searched by 5-fold stratified CV on the training window, using PR-AUC as the selection criterion. Selected settings: Logistic Regression: C=1.0; Random Forest: min_samples_leaf=100; LightGBM: num_leaves=31, scale_pos_weight=1.0; MLP: alpha=0.01. We use class weights instead of SMOTE, because SMOTE distorts probability estimates [5].

D. Calibration and operating points

Scores are recalibrated by Platt scaling [20] on the validation window. We use Platt rather than isotonic regression because only 87 validation frauds are available [21]. Expected calibration error (ECE) uses 10 equal-mass bins, because equal-width bins are almost empty at 0.2% prevalence. Two operating points are fixed on validation and then applied unchanged to test: (a) an *alert budget* of 1% of traffic, and (b) the threshold that maximises validation net benefit (Sec. IV-E). Option (b) may select "flag nothing".

TABLE III
PLANTED RISK-SCORE TERMS (POSITIVE CONTROL ONLY)

Term (binary indicator)	Weight
Night hour (00:00-05:59)	1.0
Amount above category p95 (train-fitted)	1.0
Web device	0.7
Night x high amount (interaction)	1.5
Age gap ≥ 3 brackets	0.6
Segment velocity in top 5 % (past 1 h)	0.8
P2P transfer above INR 5,000	0.6

E. INR cost model

Let r be the cost of one step-up or review (OTP, operations time, abandonment). Let $e \in [0, 1]$ be the share of a flagged fraud that the intervention prevents. The net benefit of threshold θ, relative to approving everything, is

$$B(\theta) = e \sum_i a_i \mathbb{1}[\hat{p}_i \geq \theta,\, y_i = 1] \;-\; r \sum_i \mathbb{1}[\hat{p}_i \geq \theta]. \quad (2)$$

The amounts a_i are the real recorded values, so the losses are in rupees. No currency conversion or invented per-incident loss is used. A flagged set is profitable only if its precision P satisfies

$$P \;>\; P^* = \frac{r}{e\,\bar{a}_{\text{fraud}}}, \qquad \text{required lift} = P^*/\pi, \quad (3)$$

where $\bar{a}_{\text{fraud}} =$ INR 1,504 is the mean training-window fraud amount and π is the prevalence. We take $r =$ INR 25 and $e = 1$ as the base case. The value $e = 1$ is deliberately optimistic in the model's favour. We vary r from INR 5 to INR 100. These are *stated assumptions*, not measured bank costs.

F. Positive control and CNN feature-order test

To check that the pipeline *can* detect fraud when it exists, we keep all real features and replace only the label with

$$y_i^* \sim \text{Bernoulli}\big(\sigma(\beta_0 + s\, z_i)\big), \quad (4)$$

where z_i is the standardised risk score of Table III. It includes a night×high-amount interaction that a linear model cannot represent. β_0 is solved numerically so that the mean probability equals the real prevalence of 0.192%. The strength s ranges over $\{0, 0.25, 0.5, 1, 1.5, 2\}$ with three label draws each. The case $s = 0$ reproduces a label independent of the features. The *oracle* AUC uses the true $\sigma(\beta_0 + s z_i)$. These labels are simulated and are used only to validate the method. To test the convolutional premise, we retrain the CNN on five column orderings: the canonical order and four random permutations. We compare against order-invariant references (MLP, LightGBM) at $s = 0$ (real labels) and at $s = 1.5$.

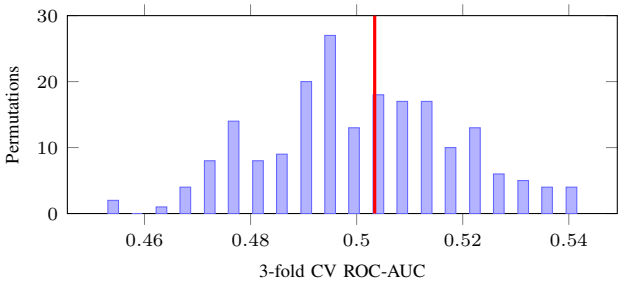


Fig. 2. Label-permutation null distribution of CV AUC (200 permutations). The red line is the true-label AUC (0.5034, $p = 0.448$).

V. Results

All results come from one run (seed 42, 2026-11-10, 45.0 minutes on 4 CPU cores). The full outputs are in the repository's `artifacts/` folder.

A. Learnability audit

Table IV lists all 18 association tests. The number with unadjusted $p < 0.05$ is 1 (`sender_age_group`). Roughly 0.9 such results are expected by chance. After Benjamini–Hochberg correction, 0 remain significant (smallest adjusted $p = 0.459$). The largest Cramér's V is 0.0080. The fraud rate across the levels of every attribute stays within a narrow band around the overall 0.192%. Jointly, the cross-validated AUC with the true labels is 0.5034. Under 200 label permutations, the null mean is 0.5005 (sd 0.0182, 95th percentile 0.5323), giving $p = 0.448$ (Fig. 2). The data therefore provide no evidence against the hypothesis that `fraud_flag` is independent of all recorded attributes.

B. Out-of-time model comparison

Table V reports the single test evaluation. Training-window CV AUCs are all close to 0.5. On the Nov–Dec test window, the learned models reach ROC-AUC between 0.495 and 0.565. 4 of 5 bootstrap 95% intervals (1,000 stratified resamples) contain 0.5. 1 unadjusted one-sided Mann–Whitney test(s) fall below 0.05, and 0 survive Holm correction for the five comparisons. The best point estimate is LightGBM (0.565). Its bootstrap interval [0.513, 0.621] has a lower bound only just above 0.5, but three observations argue against treating it as real skill. First, it is not significant once the five model comparisons are accounted for (Holm $p = 0.123$). Second, the same configuration scored 0.492 in training-window CV and 0.511 on the validation window. Third, it lies below the minimum detectable AUC of 0.582. A single nominally significant result among five, which does not replicate across time windows, is what chance alone is expected to produce. PR-AUC values are at the prevalence level (1.88×10^{-3}). Top-decile lift ranges from 0.65 to 1.30, compared with 1.0 for random ranking. Brier skill scores are essentially zero: no model predicts better than the constant base rate. Fig. 3 shows that all ROC curves follow the diagonal.

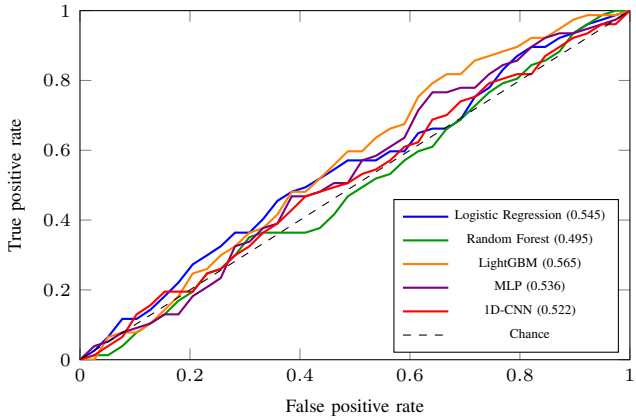


Fig. 3. Test-window ROC curves (Platt-calibrated scores). The AUC is shown in the legend.

C. Reconciled confusion counts

Table VI gives exact counts at the 1% alert budget fixed on validation. All rows sum to the 41,002 test transactions and 77 frauds. Notably, LightGBM, the model with the highest test AUC, places 0 of the 77 frauds among its 412 alerts (TP=0, FP=412, FN=77, TN=40,513). Its small AUC gain therefore does not come from the high-risk tail, where an intervention would act. Across all models, at most one fraud falls in the top 1%, against about 0.8 expected from random flagging. As an internal-consistency check, we recompute precision for every row from recall R, specificity S and prevalence π by Bayes' rule, $P = \frac{R\pi}{R\pi+(1-S)(1-\pi)}$. It matches the count-based value exactly. This is the identity that exposed the error in our first draft. Calibration matters for the class-weighted models. Logistic regression trained with balanced weights outputs raw scores near 0.5, giving ECE 475.2×10^{-3}. Platt scaling on validation reduces this to 0.43×10^{-3}. For LightGBM, ECE falls from 0.97 to 0.54 ($\times10^{-3}$). However, the calibrated probabilities are concentrated near the base rate, which is the calibrated answer when no signal exists.

D. Business impact in INR

The test window contains 77 frauds totalling INR 111,882 (mean INR 1,453). With $r =$ INR 25, the break-even precision is 1.66%, i.e. 8.7× the base rate. No model reaches this lift. At the 1% alert budget, LightGBM prevents INR 0 of fraud but incurs INR 10,300 of review cost, a net of INR -10,300. When thresholds are chosen on validation to maximise net benefit, 2 of the 5 models select "flag nothing" (Table VII). The others flag a few hundred transactions on the strength of validation noise, catch no fraud on test, and lose money. Fig. 4 shows that every model's net-benefit curve tracks the expectation for random flagging and decreases as more traffic is flagged. Table VIII varies the review cost. Even at INR 5 per review, the required lift exceeds what any model achieves, and the best realised test benefit across all costs and models is INR 0.

TABLE IV
LEARNABILITY AUDIT ON ALL 250,000 ROWS. STATISTIC: χ^2 (DOF) OR UNIVARIATE AUC (MANN–WHITNEY). RATE RANGE: MIN–MAX FRAUD RATE (%) ACROSS LEVELS.

Attribute	Statistic	Rate range (%)	Cramér's V	p	p_{BH}
transaction_type	2.32 (3)	0.183–0.239	0.0030	0.508	0.991
merchant_category	3.97 (9)	0.148–0.214	0.0040	0.913	0.991
transaction_status	0.23 (1)	0.170–0.193	0.0010	0.634	0.991
sender_age_group	11.10 (4)	0.125–0.229	0.0067	0.026	0.459
receiver_age_group	7.35 (4)	0.143–0.230	0.0054	0.118	0.991
sender_state	7.76 (9)	0.158–0.232	0.0056	0.558	0.991
sender_bank	9.20 (7)	0.161–0.250	0.0061	0.238	0.991
receiver_bank	1.82 (7)	0.164–0.207	0.0027	0.969	0.991
device_type	0.46 (2)	0.181–0.206	0.0014	0.796	0.991
network_type	2.73 (3)	0.184–0.235	0.0033	0.435	0.991
day_of_week	3.65 (6)	0.163–0.210	0.0038	0.725	0.991
hour_of_day	15.90 (23)	0.057–0.304	0.0080	0.860	0.991
is_weekend	0.31 (1)	0.189–0.200	0.0011	0.576	0.991
amount_inr	AUC_1=0.500	–	–	0.991	0.991
amount_to_cat_median	AUC_1=0.512	–	–	0.364	0.991
seg_count_1h	AUC_1=0.502	–	–	0.744	0.991
seg_count_24h	AUC_1=0.502	–	–	0.895	0.991
global_count_10m	AUC_1=0.494	–	–	0.645	0.991

TABLE V
OUT-OF-TIME TEST RESULTS (NOV–DEC 2024; 41,002 TRANSACTIONS, 77 FRAUDS). CV AUC: 5-FOLD ON TRAINING WINDOW (MEAN±SD). p: ONE-SIDED MANN–WHITNEY TEST OF AUC>0.5, RAW / HOLM-ADJUSTED. PR-AUC, BRIER SKILL AND ECE $\times 10^3$; PR-AUC OF RANDOM RANKING = 1.88.

Model	CV AUC (train)	Test ROC-AUC [95% CI]	p raw / Holm	PR-AUC	Brier skill	ECE	Lift@10%
Prevalence baseline	–	0.500 [0.500, 0.500]	–	1.88	-0.00	0.51	1.00
Logistic Regression	0.490±0.013	0.545 [0.479, 0.610]	0.086 / 0.345	2.29	-0.01	0.43	1.17
Random Forest	0.501±0.040	0.495 [0.436, 0.556]	0.558 / 0.558	1.82	-0.08	0.54	0.65
LightGBM	0.492±0.029	0.565 [0.513, 0.621]	0.025 / 0.123	2.14	-0.00	0.54	0.78
MLP	0.493±0.024	0.536 [0.472, 0.594]	0.137 / 0.410	2.10	-0.02	0.51	0.78
1D-CNN	–	0.522 [0.454, 0.582]	0.257 / 0.513	2.00	-0.03	0.51	1.30

TABLE VI
TEST CONFUSION COUNTS AT A 1% ALERT BUDGET (THRESHOLD FIXED ON VALIDATION). PREC., REC., SPEC. IN %.

Model	TP	FP	FN	TN	Prec.	Rec.	Spec.	Lift
Logistic Regression	1	384	76	40,541	0.26	1.30	99.06	1.38
Random Forest	0	376	77	40,549	0.00	0.00	99.08	0.00
LightGBM	0	412	77	40,513	0.00	0.00	98.99	0.00
MLP	1	385	76	40,540	0.26	1.30	99.06	1.38
1D-CNN	0	451	77	40,474	0.00	0.00	98.90	0.00

TABLE VII
TEST OUTCOME OF THE VALIDATION-CHOSEN COST-OPTIMAL POLICY ($r =$ INR 25, $e = 1$). $\theta = \infty$: FLAG NOTHING. AMOUNTS IN INR.

Model	θ	Flags	Caught	Prevented	Review	Net
Logistic Regression	∞	0	0	0	0	0
Random Forest	0.00294	118	0	0	2,950	-2,950
LightGBM	0.00227	598	0	0	14,950	-14,950
MLP	∞	0	0	0	0	0
1D-CNN	0.00265	451	0	0	11,275	-11,275

TABLE VIII
SENSITIVITY TO REVIEW COST r (INR). "NO-FLAG": MODELS WHOSE VALIDATION-OPTIMAL POLICY FLAGS NOTHING. NET: BEST TEST NET BENEFIT AMONG MODELS.

r	P^* (%)	Lift req.	No-flag	Best model	Net
5	0.33	1.7	1/5	MLP	0
10	0.66	3.5	1/5	MLP	0
25	1.66	8.7	2/5	Logistic Regression	0
50	3.32	17.3	3/5	Logistic Regression	0
100	6.65	34.6	3/5	Logistic Regression	0

E. Positive control

Fig. 5 and Table IX show the planted-signal experiment. At $s = 0$, all models score 0.508–0.523, reproducing the real-data result. As s grows, the models track the oracle closely. At $s = 0.5$, LightGBM reaches 0.626 against an oracle of 0.648. At $s = 1.5$, the AUCs are 0.986 (LightGBM), 0.982 (random forest) and 0.982 (logistic regression), against an oracle of 0.993. The same INR cost model then yields a positive net benefit of INR 382,551. Even the weak signal at $s = 0.5$ (oracle AUC 0.648) is already profitable (INR 42,769 for LightGBM). The break-even rule is therefore attainable whenever a real signal exists. Permutation importance on the planted setting ranks seg_count_1h, hour_of_day, device_type, age_gap, high_amount_cat highest. Each of these is an input to a planted term, so the explanation recovers the true drivers. Feature-group ablation (Table X) shows modest, consistent losses on planted labels (removing

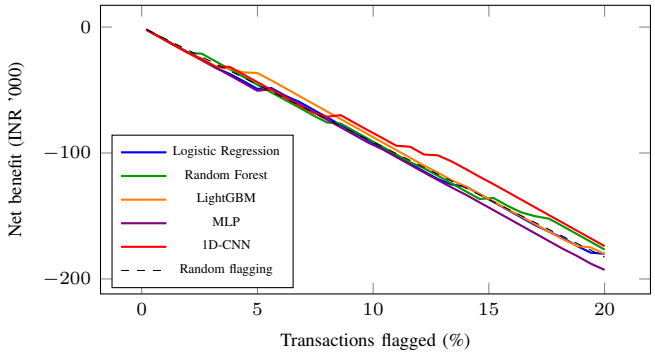


Fig. 4. Test net benefit versus share of traffic flagged ($r =$ INR 25, $e = 1$). The dashed line is the expectation for random flagging.

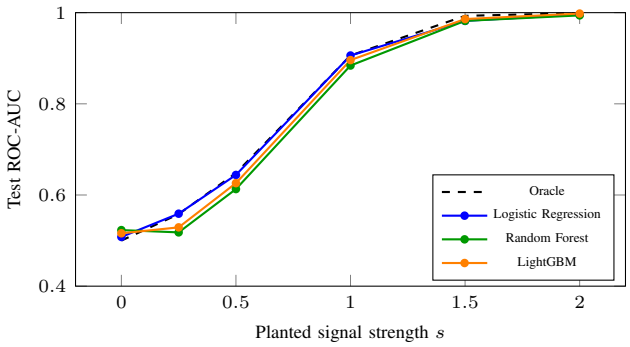


Fig. 5. Positive control on *simulated* labels (mean of three draws). The models approach the oracle once a signal exists. At $s = 0$, they reproduce the real-data result.

any group costs 0.006–0.017 AUC). The losses are small because the groups are partly redundant; for example, segment velocity encodes time of day. On the real labels, the ablation differences are within about one null standard error (0.0329) of each other, as expected for noise. The pipeline therefore has the power to find fraud signal of realistic size, and the absence of signal in the preceding subsections is a property of the dataset.

F. Does the 1D-CNN's column order matter?

Table XI shows the results. On real labels, the CNN is at chance under every ordering (0.487–0.534). On planted labels, the five orderings give almost identical ROC-AUC (0.977–0.981, sd 0.002), so column order is *not* what the network exploits. After flattening, the dense layer sees every column, and the model behaves like an MLP with a small convolutional pre-processing step. It does not outperform the order-invariant references (MLP 0.975, LightGBM 0.978). It is clearly worse on PR-AUC, the metric that matters at 0.2% prevalence (about 0.59–0.62 versus 0.68 and 0.70; Table XI). In the reported run it also took about 139 s to train, against about 3 s for LightGBM (run log). The premise of our earlier drafts, that local convolutions discover "hierarchical feature correlations" in tabular data, is therefore unsupported. This agrees with large

TABLE IX
POSITIVE CONTROL: TEST ROC-AUC (MEAN±SD, 3 LABEL DRAWS) AND LIGHTGBM NET BENEFIT (INR, $r = 25$). SIMULATED LABELS.

s	Oracle	Logistic	Rand. forest	LightGBM	Net (INR)
0.00	0.500	0.508±0.027	0.523±0.020	0.516±0.011	0
0.25	0.558	0.559±0.015	0.518±0.041	0.529±0.040	-1,559
0.50	0.648	0.644±0.029	0.613±0.017	0.626±0.015	42,769
1.00	0.906	0.906±0.024	0.884±0.023	0.896±0.019	250,392
1.50	0.993	0.982±0.010	0.982±0.015	0.986±0.009	382,551
2.00	0.999	0.996±0.004	0.994±0.004	0.998±0.001	428,144

TABLE X
FEATURE-GROUP ABLATION (LIGHTGBM TEST ROC-AUC)

Group removed	Real labels	Planted, s=1.5
None (all features)	0.565	0.978
Transaction	0.519	0.961
Temporal	0.528	0.968
Party/demographic	0.561	0.971
Device/network	0.533	0.970
Velocity (past-only)	0.524	0.964

tabular benchmarks [25], [26]. We retain the CNN only as a comparison model.

G. Latency

Scoring one transaction end to end (feature construction, model and calibration) on one CPU took a median of 16.8 ms for logistic regression and 17.1 ms for LightGBM (95th percentile 25.7 ms). The random forest took 97.8 ms, dominated by parallel-dispatch overhead. Most of the single-row time is spent constructing the pandas feature frame; a vectorised or compiled feature path would reduce it. These times exclude the network and feature-store lookups that a production authorisation path would add.

VI. DISCUSSION

A. Reconciliation with earlier versions

Table XII maps each earlier claim to its status. The earlier figures cannot be regenerated from this file under any leakage-free protocol we tested. The audit indicates why: the label carries no detectable information about the attributes. Any near-perfect score on this file must therefore come from leakage, a different dataset or a reporting error. We withdraw those claims.

B. What a deployable UPI scorer would need

The positive control shows what is missing in the data, not in the method. Card-fraud practice [6], [7] indicates that the predictive signals are entity-level. Examples are a payer's velocity and amount deviation relative to their own history, payee novelty (first payment to this VPA), device and SIM changes, collect-request patterns and beneficiary mule scores. Every one of these needs a persistent pseudonymous payer, payee and device identifier, which the public file lacks. A real study would also need the label confirmation delay, to avoid using unconfirmed recent labels [4]. It would need several

TABLE XI
1D-CNN FEATURE-ORDER SENSITIVITY (TEST ROC-AUC; PR-AUC $\times 10^3$)

Labels	Ordering / model	ROC-AUC	PR-AUC
Real	canonical	0.522	2.00
Real	random #1	0.534	2.08
Real	random #2	0.533	2.07
Real	random #3	0.520	2.04
Real	random #4	0.487	1.89
Real	MLP (order-invariant reference)	0.464	1.70
Real	LightGBM (order-invariant reference)	0.564	2.17
Planted, s=1.5	canonical	0.977	606.73
Planted, s=1.5	random #1	0.980	613.24
Planted, s=1.5	random #2	0.981	584.78
Planted, s=1.5	random #3	0.979	622.20
Planted, s=1.5	random #4	0.978	589.23
Planted, s=1.5	MLP (order-invariant reference)	0.975	676.03
Planted, s=1.5	LightGBM (order-invariant reference)	0.978	700.70

TABLE XII
EARLIER CLAIMS VERSUS THIS RUN

V1/V2 claim	Status in this run
AUC 0.9991 (1D-CNN)	Not reproducible. Test AUC 0.522; best model 0.565.
Precision = recall = 97.69%	Internally inconsistent (implied precision ≈23.6%). At a 1% alert budget, the best model catches 1 of 77 frauds (precision ≤0.26%).
Test set 31,250 *and* 625 rows	One test window: 41,002 rows, 77 frauds (Nov–Dec).
"Empirical" and "generated" data	Public, undocumented origin; label independent of attributes.
$6.35M savings at $50K/fraud	Replaced by INR model on real amounts; optimal policy saves INR 0.
Telemetry adds +0.15% AUC	On real labels, group ablation differences lie within noise (null SE 0.0329; Table X).
Latency <12 ms	Measured end to end in Python: 17.1 ms (LightGBM, median).

months of out-of-time tests; Sec. IV-B shows that 77 frauds cannot confirm an AUC below 0.582. Finally, it would need measured intervention costs and abandonment rates in place of our assumed INR 25.

C. Threats to validity

Construct: the meaning of `fraud_flag` is undocumented. *Internal:* hyper-parameter grids were small. With no signal, larger grids would only increase the risk of selection noise, and the positive control shows that the chosen settings are adequate when signal exists. *External:* the conclusions apply to this file; they do not show that UPI fraud is unpredictable. *Ethical:* age-based features could produce disparate step-up rates and must be monitored by group if used on real data [10].

VII. REPRODUCIBILITY

Code, the dataset hash, all artifacts and a Streamlit application are in the project repository: https://github.com/yogendra9476/UPI-Guard-Fraud-Detection. The following commands reproduce every table and figure.

1) `pip install -r requirements.txt` (plus `torch` for the CNN). The reported run used Python 3.13.18, scikit-learn 1.9.1, LightGBM 4.7.0, pandas 3.0.5, NumPy 2.5.3 and PyTorch 2.15.0+cu132.
2) `python run_pipeline.py`: one seeded run (seed 42), which writes `artifacts/manifest.json` with the SHA-256 of the data (`8e46a45fd12c3e9e...`), the split counts and all metrics.
3) `python build_report.py`: inserts every number in this paper from the artifacts into the LaTeX source, so the text cannot drift from the run.
4) `pytest -q`: twelve tests covering schema, chronological disjointness, absence of post-authorisation inputs, past-only velocity, train-only statistics, the precision identity and the cost model.
5) `streamlit run app.py`: an eight-page application for data exploration, the audit, results, the positive control, interactive INR cost analysis, live single and batch pre-debit scoring, and the reproducibility record.

A data card and a model card document scope, intended use and limitations [9], [10].

VIII. CONCLUSION

We rebuilt our UPI fraud study as a single traceable experiment designed around the pre-debit decision. On the supplied public dataset, a learnability audit, five learned models and an INR cost model agree. The fraud label is not reliably predictable from any pre-debit attribute. The best out-of-time AUC is 0.565 [0.513, 0.621], which is not significant after multiplicity correction and is not replicated across time windows, and the cost-optimal policy is not to intervene. A positive control shows that the same pipeline recovers near-oracle performance and positive rupee benefit when a signal exists. The 1D-CNN's convolutions provide no advantage over order-invariant models. The contribution is therefore methodological. Pre-debit feature discipline, learnability testing, power analysis, reconciled counts and break-even precision should accompany any reported fraud model, especially one trained on public data of uncertain provenance. Future work will apply the released pipeline to a labelled dataset with persistent entity identifiers and confirmed label dates.

ACKNOWLEDGMENT

We thank the course instructors and the first-review panel, whose detailed feedback motivated this reconstruction. We used scikit-learn [27], LightGBM, PyTorch, pandas, Streamlit and pgfplots. Generative AI tools were used to assist with code refactoring and language editing. All experimental results were produced by the released code and checked by the authors.

REFERENCES

[1] Reserve Bank of India, "Annual Report 2023–24," Mumbai, May 2024.
[2] National Payments Corporation of India, "UPI product statistics," 2024. [Online]. Available: https://www.npci.org.in/what-we-do/upi/product-statistics

[3] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.

[4] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, 2018.

[5] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *Proc. IEEE Symp. Comput. Intell. Data Mining (CIDM)*, 2015, pp. 159–166.

[6] C. Whitrow, D. J. Hand, P. Juszczak, D. Weston, and N. M. Adams, "Transaction aggregation as a strategy for credit card fraud detection," *Data Mining Knowl. Discov.*, vol. 18, no. 1, pp. 30–55, 2009.

[7] A. Correa Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten, "Feature engineering strategies for credit card fraud detection," *Expert Syst. Appl.*, vol. 51, pp. 134–142, 2016.

[8] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th European Modeling and Simulation Symp.*, 2016, pp. 249–255.

[9] T. Gebru *et al.*, "Datasheets for datasets," *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[10] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. ACM FAT**, 2019, pp. 220–229.

[11] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015.

[12] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd ICML*, 2006, pp. 233–240.

[13] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982.

[14] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach," *Biometrics*, vol. 44, no. 3, pp. 837–845, 1988.

[15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[16] M. Ojala and G. C. Garriga, "Permutation tests for studying classifier performance," *J. Mach. Learn. Res.*, vol. 11, pp. 1833–1863, 2010.

[17] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Stat. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.

[18] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012.

[19] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th IJCAI*, 2001, pp. 973–978.

[20] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, MIT Press, 1999, pp. 61–74.

[21] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd ICML*, 2005, pp. 625–632.

[22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[23] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in NeurIPS*, vol. 30, 2017.

[24] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[25] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in NeurIPS (Datasets and Benchmarks)*, vol. 35, 2022.

[26] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.